Day 10: Exception Handling & Modules in Python (Detailed Notes for Data Analysis Students)

Introduction

While writing programs, errors can occur due to:

- Invalid user input
- Missing files
- Division by zero
- Incorrect data types
- Network or database issues

If these errors are not handled properly, the program stops immediately. **Exception Handling** helps us manage these errors gracefully.

What is an Exception?

An **exception** is an error that occurs during program execution.

Example:

```
print(10 / 0)
```

Output:

```
ZeroDivisionError: division by zero
```

Why Use Exception Handling?

- Prevent program crashes
- Display user-friendly error messages
- Improve application reliability
- Handle unexpected situations
- Continue program execution when possible

Common Exceptions

Exception	Cause
ZeroDivisionError	Dividing by zero
ValueError	Invalid input type
TypeError	Unsupported data types
IndexError	Invalid list index
KeyError	Missing dictionary key
FileNotFoundError	File not found
NameError	Undefined variable
AttributeError	Invalid object method

---

try and except

Syntax:

```
try:
    # Risky code
```

```
except:
    # Error handling
```

Example:

```
try:
    number = 10 / 0

except ZeroDivisionError:
    print("Cannot divide by zero.")
```

Output:

```
Cannot divide by zero.
```

Handling Multiple Exceptions

```
try:

    number = int(input("Enter Number: "))
    result = 100 / number

    print(result)

except ValueError:

    print("Please enter numbers only.")

except ZeroDivisionError:

    print("Zero is not allowed.")
```

Using else

The `else` block runs only if **no exception occurs**.

```
try:

    number = int(input("Enter Number: "))
    print(100 / number)
```

```
except:

    print("Error")

else:

    print("Calculation Successful")
```

Using finally

The `finally` block always executes, whether an exception occurs or not.

```
try:

    file = open("data.txt", "r")

except:

    print("File Not Found")

finally:

    print("Program Finished")
```

Complete Example

```
try:

    a = int(input("Enter First Number: "))
    b = int(input("Enter Second Number: "))

    print(a / b)

except ValueError:

    print("Enter valid numbers.")

except ZeroDivisionError:

    print("Second number cannot be zero.")
```

```
else:
    print("Division Completed Successfully.")
finally:
    print("Thank You")
```

Raising Exceptions

You can create your own exceptions using `raise`.

```
age = 15
if age < 18:
    raise Exception("Age must be 18 or above.")
```

Custom Exception Example

```
salary = -5000
if salary < 0:
    raise ValueError("Salary cannot be negative.")
```

Modules in Python

A **module** is a Python file containing reusable functions, classes, and variables.

Example:

```
import math
```

Why Use Modules?

Modules help:

- Reuse code
- Organize projects
- Reduce development time

- Access powerful built-in functions

Importing Modules

```
import math

print(math.sqrt(25))
```

Output:

```
5.0
```

Import Specific Function

```
from math import sqrt

print(sqrt(81))
```

Output:

```
9.0
```

Math Module

```
import math

print(math.pi)
print(math.sqrt(64))
print(math.ceil(5.2))
print(math.floor(5.8))
print(math.factorial(5))
```

Output:

```
3.141592653589793
8.0
6
5
120
```

Random Module

```
import random

print(random.randint(1, 10))
```

Random output:

```
7
```

RANDOM CHOICE

```
import random

names = ["Rahul", "Amit", "Priya"]

print(random.choice(names))
```

Datetime Module

```
from datetime import datetime

today = datetime.now()

print(today)
```

CURRENT DATE

```
from datetime import date

print(date.today())
```

OS Module

```
import os

print(os.getcwd())
```

Displays the current working directory.

LIST FILES

```
import os

print(os.listdir())
```

Create Folder

```
import os

os.mkdir("Reports")
```

Rename File

```
import os

os.rename("old.txt", "new.txt")
```

Real Data Analyst Examples

EXAMPLE 1: SAFE DIVISION

```
def calculate_average(total, count):

    try:

        return total / count

    except ZeroDivisionError:

        return "No records found."

print(calculate_average(1000, 0))
```

EXAMPLE 2: EMPLOYEE AGE VALIDATION

```
age = int(input("Enter Employee Age: "))

try:
```

```
        if age < 18:
            raise Exception("Employee must be at least 18 years old.")
        print("Valid Employee")
    except Exception as e:
        print(e)
```

EXAMPLE 3: RANDOM EMPLOYEE SELECTION

```
import random
employees = ["Rahul", "Amit", "Priya", "Neha"]
print(random.choice(employees))
```

EXAMPLE 4: REPORT DATE

```
from datetime import date
print("Report Date:", date.today())
```

Mini Project 1: Safe Calculator

```
try:
    num1 = int(input("Enter First Number: "))
    num2 = int(input("Enter Second Number: "))
    print("Result =", num1 / num2)
except ZeroDivisionError:
    print("Cannot divide by zero.")
except ValueError:
    print("Enter numbers only.")
```

Mini Project 2: Lucky Employee

```
import random

employees = ["Rahul", "Amit", "Priya", "Neha"]

print("Lucky Employee:", random.choice(employees))
```

Mini Project 3: Attendance Report

```
from datetime import date

print("Attendance Report")

print("Date:", date.today())
```

Mini Project 4: Square Root Calculator

```
import math

number = int(input("Enter Number: "))

print(math.sqrt(number))
```

Practice Questions

EXCEPTION HANDLING

1. Handle division by zero.
2. Handle invalid input.
3. Handle missing file.
4. Handle invalid list index.
5. Handle missing dictionary key.
6. Use `else`.
7. Use `finally`.
8. Raise custom exception.
9. Validate age.
10. Validate salary.

MODULES

11. Find square root.
12. Find factorial.
13. Print PI value.
14. Generate random number.
15. Select random employee.
16. Print current date.
17. Print current time.
18. Display working directory.
19. List all files.
20. Create a new folder.

BUSINESS PROBLEMS

21. Safe salary calculator.
22. Employee age validation.
23. Invoice generator with current date.
24. Random prize winner.
25. Attendance report.
26. Report folder creation.
27. File validation.
28. Safe average calculation.
29. Employee ID generator.
30. Sales report generator.

Assignment (Data Analyst Level)

Create an **Employee Salary Analyzer**.

Employee Salaries:

```
salaries = [50000, 60000, 55000, 70000]
```

Requirements:

1. Calculate average salary safely.
2. Handle division by zero using `try-except`.
3. Print today's date.

4. Save report in a folder named `Reports`.
5. Include report generation time.

Interview Questions

BASIC

1. What is an exception?
2. Why do we use exception handling?
3. Difference between syntax error and runtime error?
4. What is `try`?
5. What is `except`?

INTERMEDIATE

6. What is `else`?
7. What is `finally`?
8. What is `raise`?
9. Difference between `Exception` and `ValueError`?
10. How do you handle multiple exceptions?

MODULES

11. What is a module?
12. Difference between module and package?
13. What is the `math` module?
14. What is the `random` module?
15. What is the `datetime` module?
16. What is the `os` module?
17. How do you import a specific function?
18. Why are modules useful?
19. How do you create your own module?
20. Give a real-world example of module usage in Data Analysis.

Real-Time Interview Scenarios

SCENARIO 1

A CSV file is missing. How will you ensure the program does not crash?

Answer: Use `try-except` to catch `FileNotFoundError` and display an appropriate message.

SCENARIO 2

A user enters `"ABC"` instead of a number.

Answer: Catch the `ValueError` exception and ask the user to enter a valid number.

SCENARIO 3

Generate a daily report with the current date and time.

Answer: Use the `datetime` module.

SCENARIO 4

Select a random employee for the "Employee of the Month" award.

Answer: Use `random.choice()`.

SCENARIO 5

Create a report folder if it does not already exist.

Answer:

```
import os

if not os.path.exists("Reports"):
    os.mkdir("Reports")
```

Learning Outcome

After Day 10, you will be able to:

✅ Handle runtime errors using `try-except`

✅ Use `else`, `finally`, and `raise` effectively

✅ Work with Python's built-in modules (`math`, `random`, `datetime`, `os`)

✅ Build more reliable and professional Python programs

✅ Solve common Data Analyst interview scenarios involving error handling and automation